

Lecture 2

The basic logical functions

- The digital basic logic functions
- The industrial PLC programming standard IEC61131-3
- Combination of logics
- Functions and Function Blocks
- Declaring variables

Logical expression

- Logic can be expressed by
 - Relay scheme
 - Logic scheme
 - Boolean Algebra
- In IEC 61131-3 are these schemes represented by following programming languages
 - Ladder (LD)
 - Function Block Diagram (FBD)
 - Structure Text (ST)

The three logical basic (and only) functions

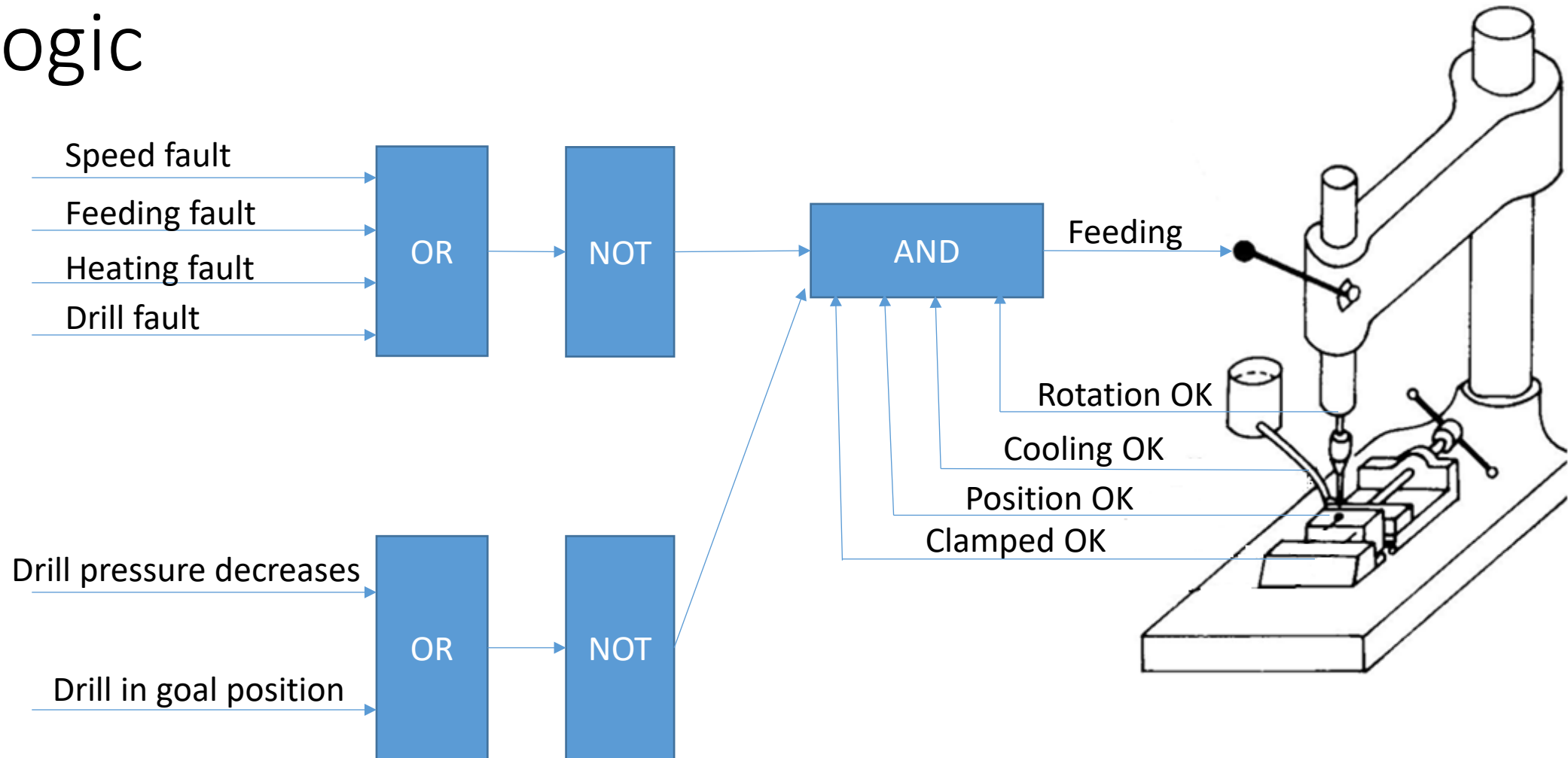
(George Bool 1854)

AND

OR

NOT

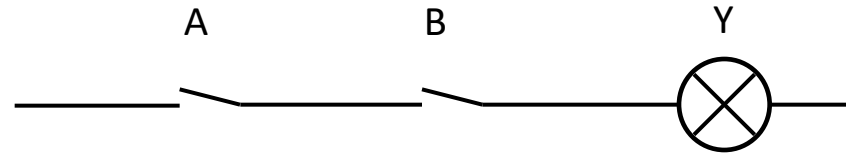
Logic



AND

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

- El-scheme:



- Logic scheme:



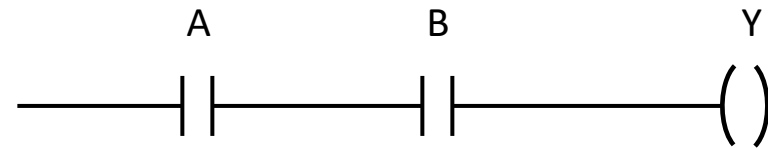
- Boolean algebra: $A * B = Y$

$$A \wedge B = Y$$

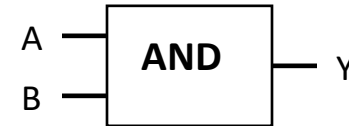
AND (IEC 61131-3)

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

- Ladder (LD):



- Function Block Diagram (FBD):



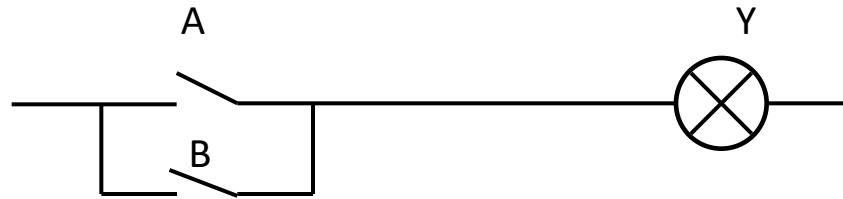
- Structure Text (ST):

Y := A **AND** B;

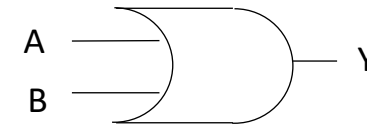
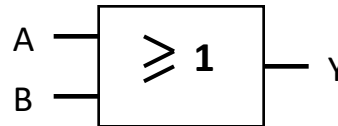
OR

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

- El-scheme:



- Logic scheme:



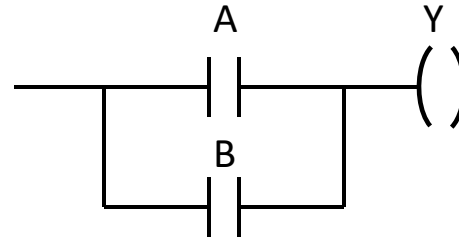
- Boolean algebra: $A + B = Y$

$$A \vee B = Y$$

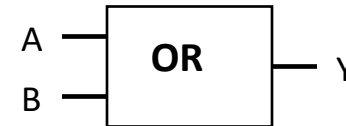
OR (IEC 61131-3)

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

- Ladder (LD):



- Function Block Diagram (FBD):



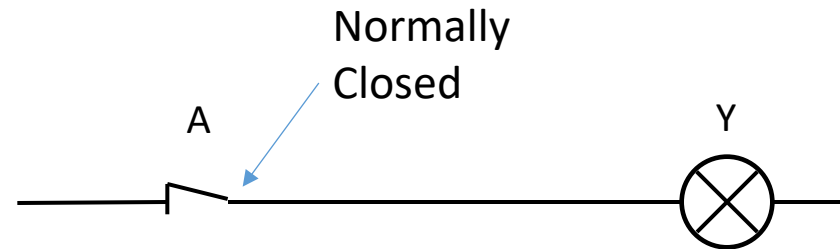
- Structure Text (ST):

Y := A OR B;

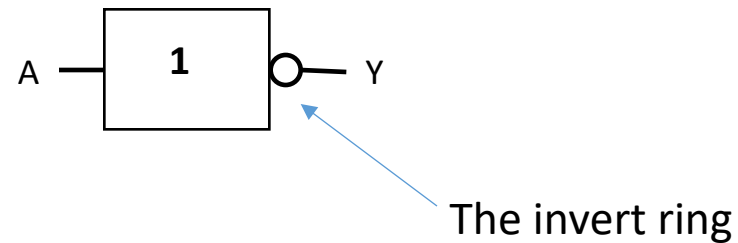
NOT

A	Y
0	1
1	0

- El-scheme:



- Logic schema:



- Boolean algebra:

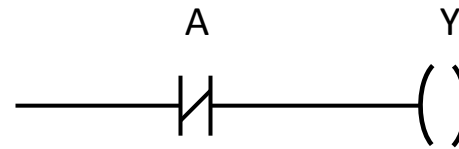
$$A = \overline{Y}$$

$$A = \neg Y$$

NOT (IEC 61131-3)

A	Y
0	1
1	0

- Ladder (LD):



- Function Block Diagram (FBD):



- Structure Text (ST):

$Y := \text{NOT } A;$

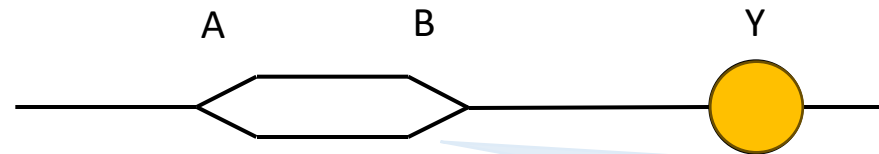
Combination of logic

- There are only three logical functions (AND, OR and NOT)
- The rest is combinations

XOR (Exclusive OR)

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

- El-scheme:



- Logic scheme:



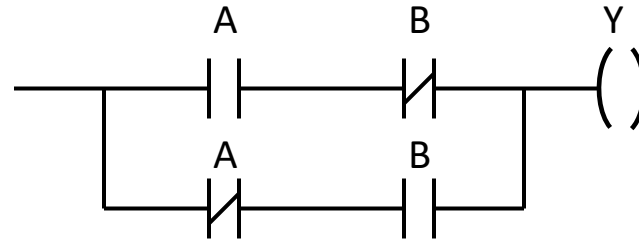
Staircase wiring
circuit
(Trappomkopplare)

- Boolean algebra: $Y = A \oplus B$

XOR (IEC 61131-3)

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

- Ladder (LD):



- Function Block Diagram (FBD)



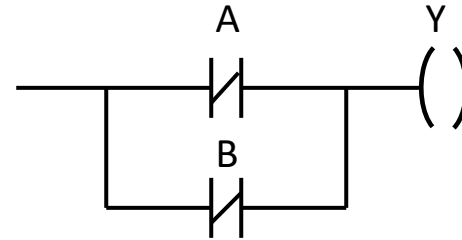
- Structure Text (ST):

$Y := A \text{ XOR } B;$

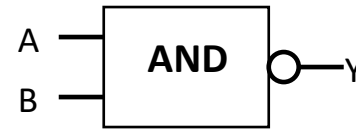
NAND

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

- Ladder (LD):



- Function Block Diagram (FBD):



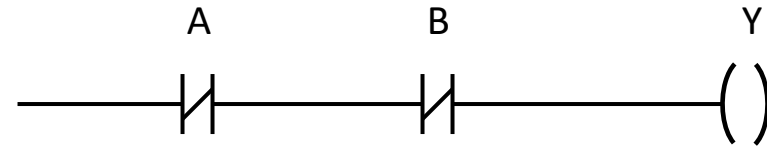
- Structure Text (ST):

$Y := \text{NOT}(A \text{ AND } B);$

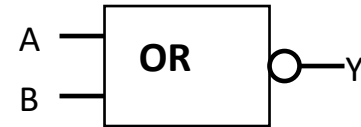
NOR

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- Ladder (LD):



- Function Block Diagram (FBD):



- Structure Text (ST): $Y := \text{NOT } (A \text{ OR } B);$

Boolean algebra translated to Structure Text

Boolean Algebra	Structure Text (ST)
$A * B$	A AND B
$A + B$	A OR B
$A = B$	A := B
$\overline{A}$	NOT A
	; End of the expression

- Structure Text has the same mathematical roles as Boolean algebra
- AND priorities before OR

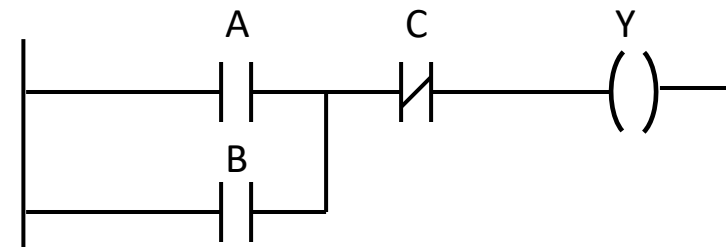
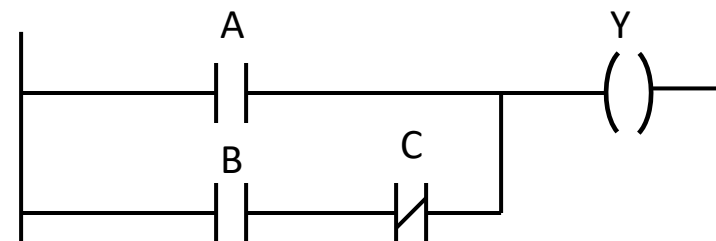
Priority of Combined Expression

Structure Text

$Y := A \text{ OR } B \text{ AND NOT } C;$

$Y := (A \text{ OR } B) \text{ AND NOT } C;$

Ladder



AND priorities before OR

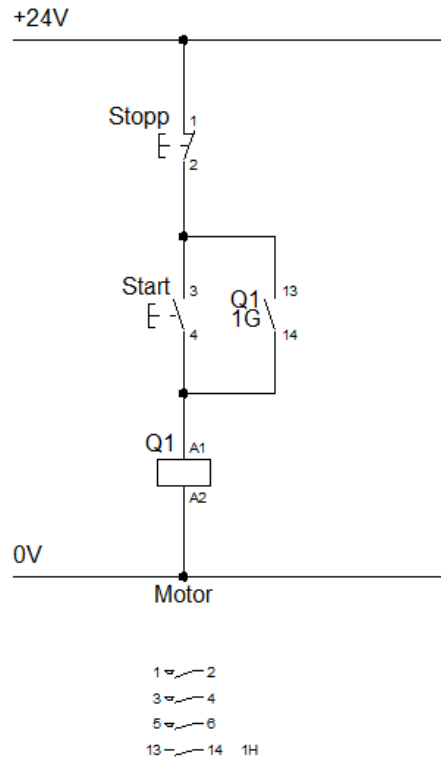
Variables types and variable name

Type	IEC.	Comment
Input	IX	I=Input X=Bit
Output	QX	Q=Output X=Bit
Memory	MX	M=Memory X=Bit
Timer	TON	Standard Function block
Counter	CTU	Standard Function block

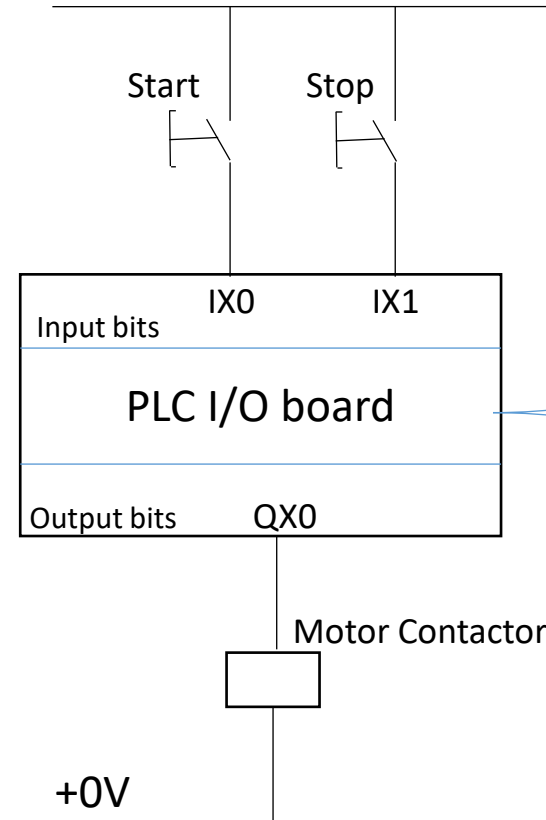
A start button is connected to an input proposed to be named ixStartButton and declared in a list called HEADER between VAR and END_VAR.

Memory function / Self holding

Scheme of an electrical contactor solution



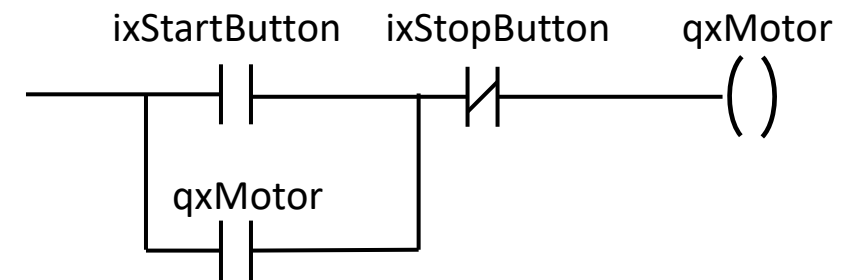
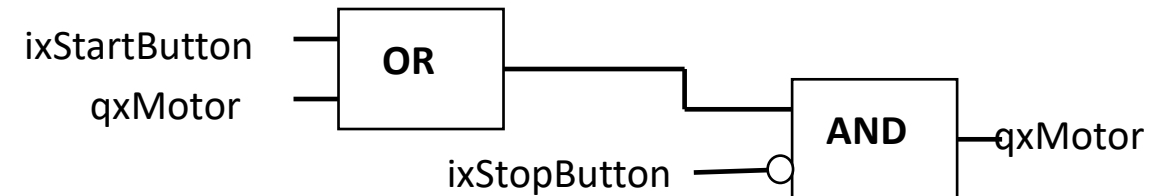
+24VDC



VAR

IX0 ixStartButton
IX1 ixStopButton
QX0 qxMotor

END_VAR

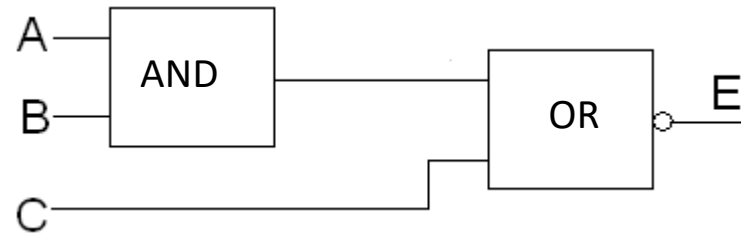


De Morgan's teorem

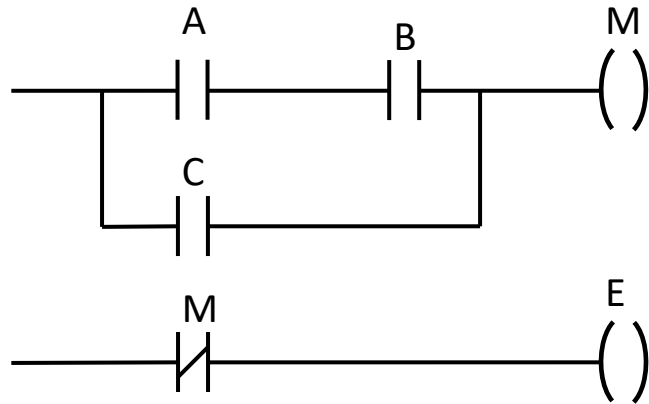
NOT (P AND Q) = NOT P OR NOT Q

NOT (P OR Q) = NOT P AND NOT Q

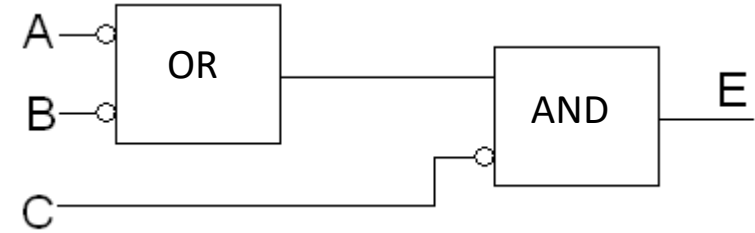
Example of de Morgan (All alternatives gives the same result)



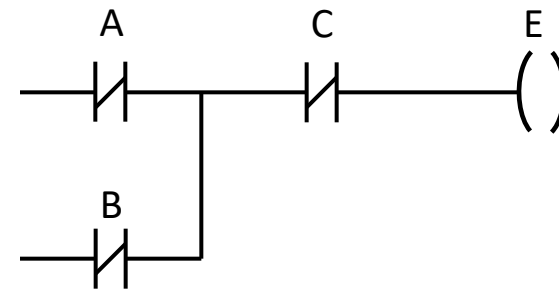
$E := \text{NOT}(A \text{ AND } B \text{ OR } C);$



A	B	C	E
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

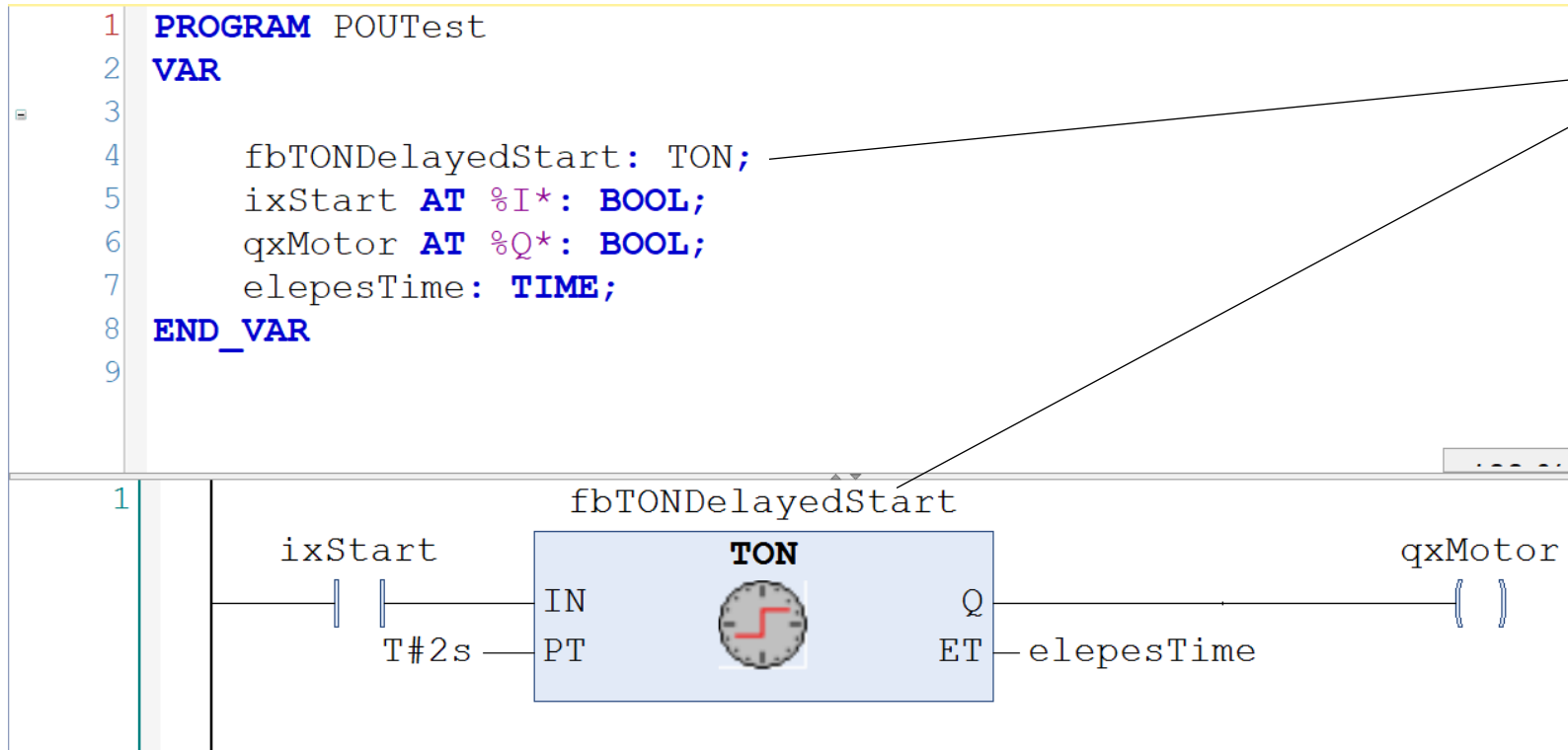


$E := (\text{NOT } A \text{ OR } \text{NOT } B) \text{ AND } \text{NOT } C;$



Function Block, Timer on (TON)

- DATE TIME are special variable types for dates and time
- TIME type is written T#5h2m3s



Function blocks must be declared in the header list which gives them a space in the memory

- When ixStart and IN become TRUE, the Q and qxMotor will become TRUE after 2s
- When ixStart and IN becomes FALSE the Q and qxMotor will immediately be FALSE

Function block, Counter Up (CTU)

- CTU count up CV and the WORD variable mwCounterValue with 1 each time CU turn TRUE by ixCount
- When CV = PV the Q and qxLedCounted will turn TRUE
- ixReset will reset the CTU to 0 CV will be equal to 0

```
1 PROGRAM POUTest
```

```
2 VAR
```

```
3     ixCount          AT %I*: BOOL;
```

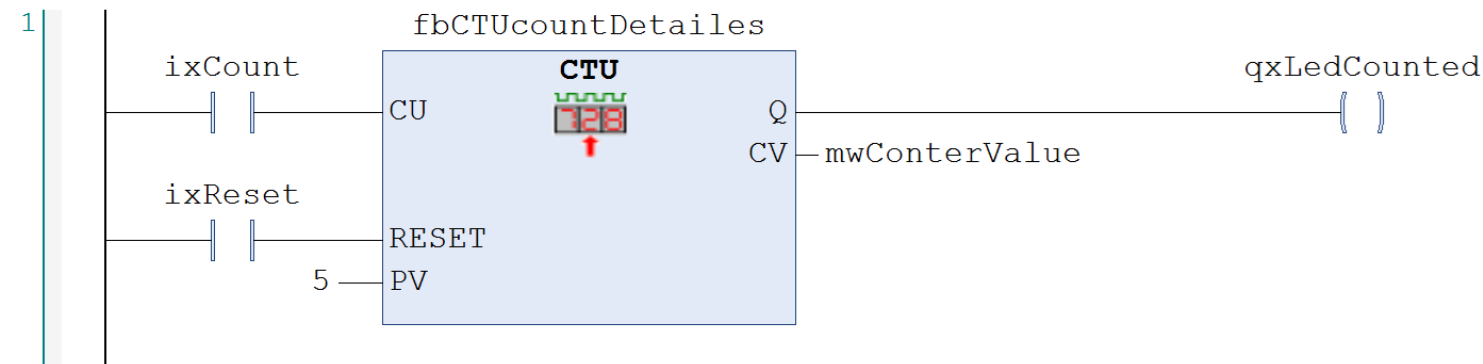
```
4     ixReset          AT %I*: BOOL;
```

```
5     qxLedCounted     AT %Q*: BOOL;
```

```
6     fbCTUcountDetailes: CTU;
```

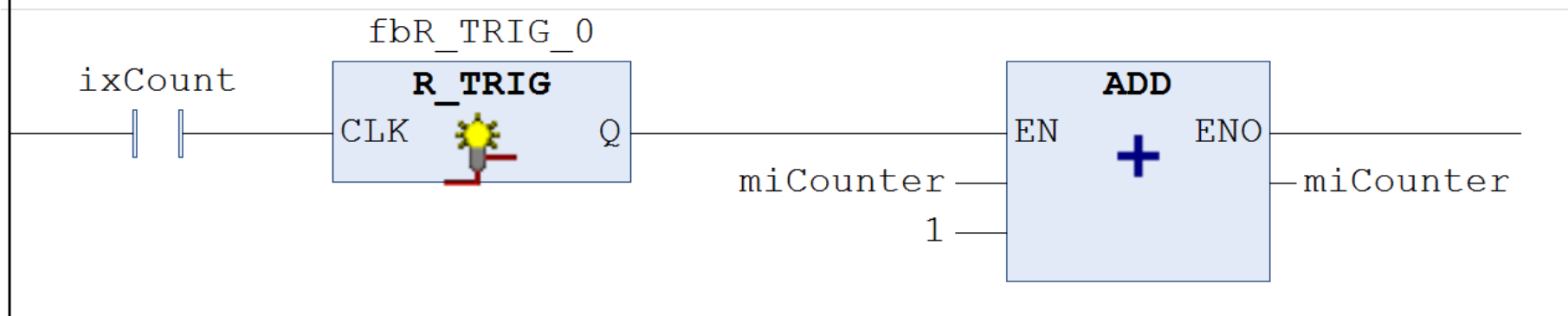
```
7     mwConterValue: WORD;
```

```
3 END_VAR
```



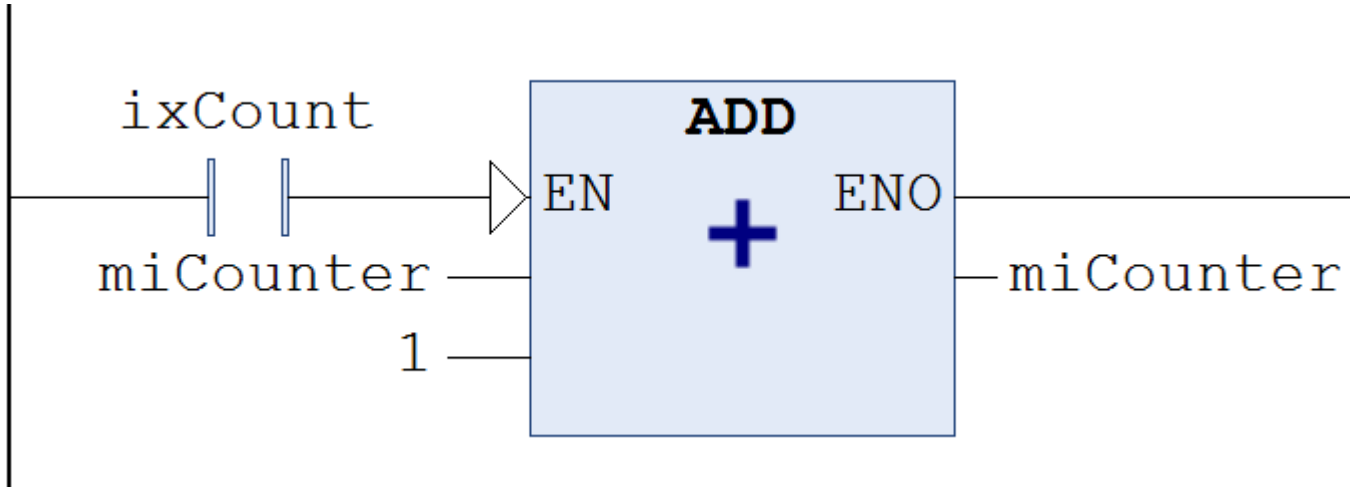
One scan pulse R_TRIG (Edge detection)

```
9  
10 fbR_TRIG_0: R_TRIG;  
11 miCounter: INT;  
12 END_VAR  
13
```



- Alternative to CTU
- The integer variable miCounter will count up with 1
- miCounter should count up unlimited without R_TRIG when ixCount is TRUE

An alternative to R_TRIG works in TwinCat

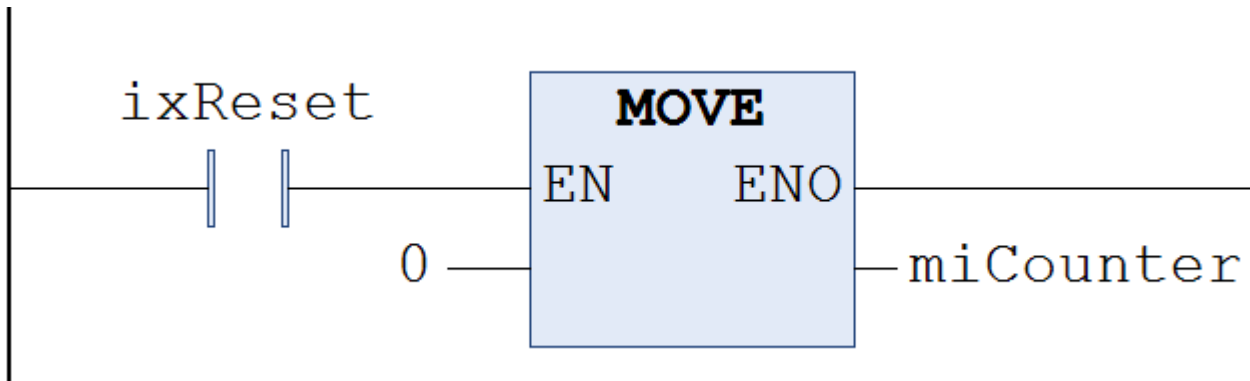


Structure text

```
IF ixCount AND NOT mxCount THEN  
    miCounter := miCounter + 1;  
END_IF  
mxCount := ixCount;
```

- Alternative to CTU
- The integer variable miCounter will count up with 1
- Right click on EN and select Edge detection

Assign a variable



Structure text

```
IF ixReset THEN  
    miCounter := 0;  
END_IF
```

- MOVE is here used to reset miCounter

Declare variable

VAR

<Identifier> {AT <Address>}: <Type> {:=<default value>} ;

END_VAR;

optional

(* Example *)

VAR

ixRun AT %I* : BOOL;

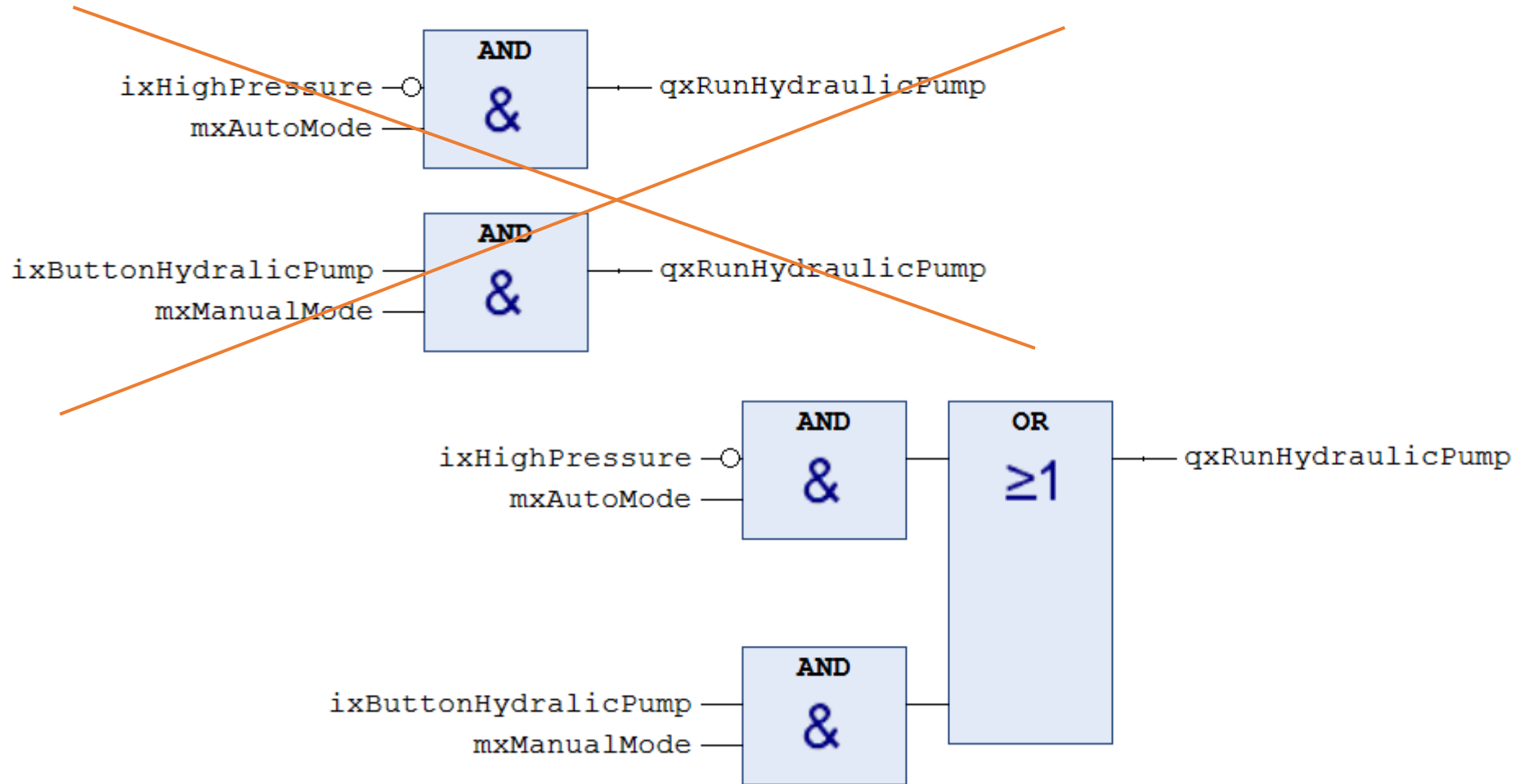
mxStop : BOOL;

miPosition : INT := 10; (* Default position *)

END_VAR;

* Connects to physical input in the solution explorer

Never assign the same variable twice!



Chapter 4 of the book Programmable Logic Controller

Dag H. Hansen

- Read through chapter 3 Analog signal
- Read chapter 4
- 4.3 – 4.4 digital logic and Boolean design
- Test Problems: 4.1- 4.9